



COMP 2067

Introduction to

Formal Reasoning

Lecture 10 Lists

Author: Prof. Thorsten Altenkirch

Adapted by: Dr. Kweh Yeah Lun



Learning Outcomes

At the end of this lecture, students will be able to

- define the inductive type: list
- explain and prove the basic properties of lists
- understand the usage of tactic: injection
- understand how insertion sort is being defined in Lean



Introduction

- Given $A : \text{Type}$, there is $\text{list } A : \text{Type}$ which is the type of **finite sequences of elements of A** .
 - Example: $[1,2,3] : \text{list } \mathbb{N}$ or $[tt] : \text{list } \text{bool}$.
- Can also have **lists of lists**.
 - Example: $[[],[0],[0,1],[0,1,2]] : \text{list } (\text{list } \mathbb{N})$ is a **list of lists of numbers**.
- All elements of a list must have the same type, hence something like $[1,tt]$ is not permitted.



- For any **type** A : can construct **empty list**: $\text{list } [] : \text{list } A$, which is also called **nil**.
- If there is an element $a : A$ and a **list** $l : \text{list } A$, then the list $a :: l : \text{list } A$ can be formed with a in front of l
- Example: $1 :: [2,3] = [1,2,3]$.
- Also known as **cons**.
- Example: $[1,2,3] = 1 :: 2 :: 3 :: []$ or using **nil** and **cons**, which is **cons 1 (cons 2 (cons 3 nil))**



- **All lists can be created from nil and cons**
- Hence lists can be defined inductively in a manner very similar to $\mathbb{N}$

Try it

```
inductive list (A : Type)
| nil  : list
| cons : A → list → list
```

- This declaration means:
 - For any type $A : \text{Type}$, there is a new type $\text{list } A : \text{Type}$
 - There are elements $\text{nil} : \text{list } A$, and given $a : A$ and $l : \text{list } A$, then function $\text{cons } a \ l$.
 - All the elements of $\text{list } A$ can be generated by nil and then applying cons a finite number of times,
 - **nil and $\text{cons } a \ l$ are different elements,**
 - **cons is injective,** i.e. given **$\text{cons } a \ as = \text{cons } b \ bs$, then $a = b$ and $as = bs$.**



Basic Properties of list

- Most of the basic properties and their proofs are very similar to the natural numbers.
- First: show that $\text{nil} \neq a :: l$ which corresponds to $0 \neq \text{succ } n$ and $\text{true} \neq \text{false}$.
- These properties are called **no confusion properties**.
- Use **contradiction** as before.

```
theorem no_conf :  $\forall (a:A)(l : \text{list } A), \text{nil} \neq a :: l :=$   
begin  
  assume a l,  
  contradiction  
end
```

[Try it!](#)



- Show: **injectivity** of $::$
- Unlike in the case for natural numbers, there are two results to prove: given that $a :: l = a' :: l'$, show that $a = a'$ and $l = l'$.
- For the second part, define **tl** (for tail) and use this in the proof:

```
def tl : list A → list A
| [] := []
| (a :: l) := l

theorem inj_tl : ∀ (a a' : A) (l l' : list A), a :: l = a' :: l' → l = l' :=
begin
  assume a a' l l' h,
  change tl (a :: l) = tl (a' :: l'),
  rewrite h,
end
```

Try it!



- For the first part, it can be done by using tactic **injection**.

```
theorem inj_hd :  $\forall$  (a a':A)(l l' : list A), a :: l = a' :: l'  $\rightarrow$  a = a' :=  
begin  
  assume a a' l l' h,  
  injection h,  
end
```

Try it!

- **injection** also works in the proof of **inj_tl** alleviating the need to define **tl**.



Tactic: injection

- If two values of an inductive type, built with the same constructor, are equal, then the only way this can happen is if their constituent parts (the arguments to the constructor) are also equal.
- This is a fundamental property of how inductive types are defined.
- injection automates this reasoning step, no need to manually apply congruence lemmas for each argument.



Insertion Sort

- Inefficient algorithm with quadratic complexity while the best sorting algorithms have a $n \log n$ complexity.
- However, insertion-sort is especially easy to implement and verify.
- For simplicity, sort lists of natural numbers with respect to $\leq$
 $: \mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Prop}$.



- Goal: implement a function **sort : list $\mathbb{N}$ $\rightarrow$ list $\mathbb{N}$** that sorts a given list,
Example: **sort [6,3,8,2,3] = [2, 3, 3, 6, 8]**.
- Insertion sort can be easily done with a deck of cards:
 - start with an unsorted deck and take a card from the top each time and insert it at the appropriate position in the already sorted pile.
 - Inserting a card means go through the sorted pile until a card that is greater is found and then insert the card just before that.
 - There is also a special case that the sorted pile is empty in which case insert directly and a pile out of one card is made.



- Here is the recursive function implementing the algorithm:

```
def ins : N → list N → list N
| a [] := [a]
| a (b :: bs) := if ble a b then a :: b :: bs else b::(ins a bs)

#reduce ins 6 [2, 3, 3, 8]
```

Try it!

- Using **ins**, it is easy to implement **sort** by inserting one element after the other.

```
def sort : list N → list N
| [] := []
| (a :: as) := ins a (sort as)

#reduce (sort [6,3,8,2,3])
```

Try it!



$\text{ins } 6 \ [2,3,3,8] = \text{ins } 6 \ (\underline{2} :: [3,3,8])$
 $= \text{if } \underline{\text{ble } 6 \ 2} = \underline{\text{ble } 5 \ 1} = \underline{\text{ble } 4 \ 0} = \text{ff},$
 $\text{else } \underline{2} :: (\text{ins } 6 \ [3,3,8])$
 $= \text{ins } 6 \ (\underline{3} :: [3, 8])$
 $\text{if } \underline{\text{ble } 6 \ 3} = \underline{\text{ble } 5 \ 2} = \underline{\text{ble } 4 \ 1} = \underline{\text{ble } 3 \ 0} = \text{ff},$
 $\text{else } \underline{3} :: (\text{ins } 6 \ [3,8])$
 $= \text{ins } 6 \ (\underline{3} :: [8])$
 $\text{if } \underline{\text{ble } 6 \ 3} = \underline{\text{ble } 5 \ 2} = \underline{\text{ble } 4 \ 1} = \underline{\text{ble } 3 \ 0} = \text{ff},$
 $\text{else } \underline{3} :: (\text{ins } 6 \ [8])$
 $= \text{ins } 6 \ (\underline{8} :: [])$
 $\text{if } 6 \ 8 = \underline{\text{ble } 5 \ 7} = \underline{\text{ble } 4 \ 6} = \underline{\text{ble } 3 \ 5} = \underline{\text{ble } 2 \ 4} = \underline{\text{ble } 1 \ 3} = \underline{\text{ble } 0 \ 2} = \underline{\text{tt}}$
 $\underline{6} :: 8 :: [] = [6,8]$

$[2,3,3,6,8]$



- Give yourself a big clap.
- Survived once again? Great! ◀◀
- Any question?



University of
Nottingham

UK | CHINA | MALAYSIA

Next lecture...

- Introduction to trees



“There are only two kinds of programming languages: those people always complain about and those nobody uses.”

– Bjarne Stroustrup

Hope you all enjoy this lecture!